Reverse Engineering: x86-64 Assembly

2026-10-13

Table of contents

1 Introduction **1**

2 Install NASM **2**

3 Assemble and Run **2**

4 Program Structure **2**

5 Part 1: Hello World **3**

6 Part 2: Registers **4**

7 Part 3: mov **4**

8 Part 4: Arithmetic **6**

9 Part 5: Comparisons and Jumps **6**

10 Part 6: Loops **8**

1 Introduction

Reading disassembled code is much easier once you have written some yourself. In this lab you will write and run a series of short x86-64 assembly programs using NASM. The instructions, registers, and patterns you use here are exactly what you will encounter when analyzing compiled binaries in later labs.

Complete this lab on your virtual machine.

2 Install NASM

NASM (Netwide Assembler) is the assembler you will use to turn assembly source files into executable programs.

```
sudo apt update
sudo apt install nasm
```

Confirm the installation:

```
nasm --version
```

3 Assemble and Run

Throughout this lab you will assemble and run small programs. The process is always the same: two commands to build the program and one to run it. Given a source file named `program.asm`:

```
nasm -f elf64 program.asm -o program.o
ld program.o -o program
./program
```

- `nasm -f elf64` assembles the source into a 64-bit ELF object file.
- `ld` links the object file into an executable.
- `./program` runs it.

4 Program Structure

Every NASM program for this lab follows the same skeleton:

```
section .data
    ; named data goes here

section .text
    global _start

_start:
    ; instructions go here
```

- `section .data` is where you define named values and strings.
- `section .text` holds the instructions.
- `_start` is the entry point — execution begins here.
- Every program must end by calling the `exit` syscall (shown below), otherwise it will crash.

5 Part 1: Hello World

Create a file named `hello.asm` with the following contents:

```
; hello.asm

BITS 64 ; not a required line. This is useful for vscode extensions.

section .data
    msg db "Hello, World!", 10
    len equ $ - msg

section .text
    global _start

_start:
    mov rax, 1          ; syscall number: write
    mov rdi, 1          ; file descriptor: stdout
    mov rsi, msg        ; address of the string
    mov rdx, len        ; number of bytes to write
    syscall

    mov rax, 60         ; syscall number: exit
    mov rdi, 0          ; exit code
    syscall
```

Assemble and run it. You should see `Hello, World!` printed to the terminal. Use the following commands to assemble and run it:

```
nasm -f elf64 hello.asm -o hello.o
ld hello.o -o hello
./hello
```

What is a Syscall?

A syscall is a request to the operating system to perform an action your program cannot do on its own, such as writing to the terminal or exiting cleanly. You place the syscall number in `rax`, place the arguments in `rdi`, `rsi`, `rdx` (in that order), and then execute the `syscall` instruction. The OS handles the rest.

db and equ

`db` (define byte) declares raw bytes in the data section. The value `10` at the end is the ASCII code for a newline character.
`equ` defines a constant. `$ - msg` means "current position minus the address of msg," which equals the number of bytes in the string.

6 Part 2: Registers

Registers are small storage locations built into the CPU. In x86-64 there are sixteen general-purpose registers. You will work with four of them most often:

Register	Common use
`rax`	return values, syscall number
`rdi`	first argument
`rsi`	second argument
`rdx`	third argument

Each register has smaller sub-registers that refer to the lower bits of the same storage:

64-bit	32-bit	16-bit	8-bit
`rax`	`eax`	`ax`	`al`
`rdi`	`edi`	`di`	`dil`

When compiled C code returns an `int` from a function, the value is in `eax`, that is, the 32-bit portion of `rax`. You will see this frequently in later labs.

7 Part 3: mov

`mov` copies a value into a register or memory location. It is the most common instruction in any compiled binary.

```
mov rax, 42          ; put the number 42 into rax
mov rbx, rax         ; copy the value in rax into rbx
mov rax, 0           ; overwrite rax with 0
```

`mov` never modifies the source; it only copies.

Create a file named `mov.asm`. Experiment with moving values between registers. To observe the result, pass a register value as the exit code and inspect it with `echo $?` after running the program:

```
; mov.asm
BITS 64
section .text
    global _start

_start:
    mov rax, 7
    mov rbx, rax
    mov rax, 3
    ; at this point: rax = 3, rbx = 7

    mov rdi, rbx        ; exit code = 7
    mov rax, 60
    syscall
```

```
nasm -f elf64 mov.asm -o mov.o
ld mov.o -o mov
./mov
echo $?
```

`echo $?` prints the exit code of the last command. Use this throughout the lab to check register values.

> **Exit Codes are One Byte**
>
> Exit codes are read as a single byte (0–255). If you put a value larger than 255 in `rdi` before exiting, only the lowest 8 bits will be reported by `echo $?`.

8 Part 4: Arithmetic

The basic arithmetic instructions operate on registers directly.

Instruction	Effect
`add rax, 5`	`rax = rax + 5`
`sub rax, 3`	`rax = rax - 3`
`inc rax`	`rax = rax + 1`
`dec rax`	`rax = rax - 1`

Create a file named `arithmetic.asm`. Compute a value and verify it with `echo $?`:

```
; arithmetic.asm
BITS 64

section .text
    global _start

_start:
    mov rax, 10
    add rax, 5      ; rax = 15
    sub rax, 3      ; rax = 12
    inc rax         ; rax = 13

    mov rdi, rax
    mov rax, 60
    syscall
```

```
nasm -f elf64 arithmetic.asm -o arithmetic.o
ld arithmetic.o -o arithmetic
./arithmetic
echo $?
```

Exercise: Modify `arithmetic.asm` so that the exit code is `20`. Use at least three arithmetic instructions to arrive at that value.

9 Part 5: Comparisons and Jumps

`cmp` subtracts one value from another and records the result in status flags — but it does not store the result anywhere. The conditional jump instructions that follow it read those flags to

decide whether to jump.

```
cmp rax, 10      ; compare rax against 10
je  equal        ; jump to label "equal" if rax == 10
jne not_equal    ; jump if rax != 10
jg  greater      ; jump if rax > 10
jl  less         ; jump if rax < 10
jge ge           ; jump if rax >= 10
jle le           ; jump if rax <= 10
```

In practice only one of these jumps would follow a given `cmp`; they are listed together here for reference.

> **Signed vs. Unsigned Jumps**
>
> `jg`, `jl`, `jge`, and `jle` treat the values as **signed** integers. Compilers use `ja` (above), `jb` (below), `jae`, and `jbe` for **unsigned** comparisons. They behave the same way for small positive numbers, so expect to see both families in Cutter.

> **This is What You Will See in Cutter**
>
> When you open a compiled binary in Cutter and see a `cmp` followed by a `je` or `jne`, this is exactly what is happening: the program is checking a condition and branching based on the result. Modifying that jump instruction changes which path the program takes.

Create a file named `compare.asm`. The program checks a value and exits with a different code depending on the result:

```
; compare.asm
BITS 64

section .text
    global _start

_start:
    mov rax, 15

    cmp rax, 10
    jg  above_ten       ; if rax > 10, jump

    mov rdi, 0          ; rax <= 10: exit code 0
    mov rax, 60
```

```
    syscall

above_ten:
    mov rdi, 1          ; rax > 10: exit code 1
    mov rax, 60
    syscall
```

```
nasm -f elf64 compare.asm -o compare.o
ld compare.o -o compare
./compare
echo $?
```

Run it and check the exit code. Then change the initial value of `rax` to `5` and run it again to confirm the other branch is taken.

10 Part 6: Loops

A loop in assembly is a block of instructions with a conditional jump that points backward. The program repeats the block until the condition is no longer true.

This structure, a block with a backward edge, is exactly what you will see in Cutter when a compiled binary contains a `for` or `while` loop.

Create a file named `loop.asm`. This loop accumulates a sum:

```
; loop.asm
BITS 64

section .text
    global _start

_start:
    mov rcx, 0      ; i = 0
    mov rax, 0      ; sum = 0

loop_start:
    cmp rcx, 4      ; if i == 4, exit loop
    je  done

    add rax, 5      ; sum += 5
```

```
    inc rcx          ; i++
    jmp loop_start

done:
    mov rdi, rax     ; exit code = sum (should be 20)
    mov rax, 60
    syscall
```

```
nasm -f elf64 loop.asm -o loop.o
ld loop.o -o loop
./loop
echo $?
```

The exit code should be `20`.

Exercise: Modify the loop so it runs 5 times and adds 3 each iteration. What exit code do you expect? Verify with `echo $?`.

> **Connecting to Key Validation**
>
> In a later lab you will reverse engineer a binary that validates a license key. Part of the validation is a loop that reads digits from the key, accumulates their sum, and checks whether it equals a specific value. The loop you just wrote follows the same structure: a counter, a comparison at the top, an accumulate step, an increment, and a backward jump.
> When you see this pattern in Cutter, you will know exactly what it is doing.